

План собеседования Middle+ Go разработчика

Цель интервью

Оценить, способен ли кандидат уровня Middle+ самостоятельно проектировать и реализовывать production-grade сервисы на Go, принимать инженерные решения, находить компромиссы между простотой и масштабируемостью, разбираться в существующем коде, работать с конкурентностью, базами данных, API, тестированием и эксплуатацией сервисов.

Ожидаемый профиль Middle+

Кандидат уровня Middle+ обычно:

- уверенно пишет идиоматичный Go и понимает стандартную библиотеку;
- умеет проектировать HTTP/gRPC-сервисы, API-контракты и внутренние пакеты;
- понимает goroutines, channels, context, race conditions и базовые паттерны конкурентности;
- работает с SQL/NoSQL, транзакциями, индексами и миграциями;
- пишет unit, integration и table-driven тесты;
- способен диагностировать performance-проблемы и production-инциденты;
- понимает observability: logs, metrics, tracing, alerts;
- умеет обсуждать архитектурные trade-off и защищать технические решения;
- может ревьюить код и быть владельцем части сервиса без постоянного контроля.

Рекомендуемая структура интервью

Общая длительность: 90-120 минут.

Блок	Длительность	Что проверяем
1. Вводная часть и опыт	10 мин	Релевантность опыта, зрелость, зона ответственности
2. Go core и идиоматика	20 мин	Знание языка, стандартной библиотеки, ошибок, интерфейсов
3. Concurrency и context	20 мин	Goroutines, channels, cancellation, races, deadlocks
4. Backend, API и данные	20 мин	HTTP/gRPC, БД, транзакции, надежность
5. Практическая задача	25-35 мин	Код, декомпозиция, тестируемость, edge cases
6. System design light	20 мин	Архитектурное мышление, компромиссы, эксплуатация
7. Поведенческий блок и вопросы кандидата	10 мин	Коммуникация, ownership, работа в команде

1. Вводная часть и опыт

Цели блока

- Понять, какие задачи кандидат реально решал сам.
- Выяснить размер и критичность систем, с которыми он работал.
- Отличить практический production-опыт от поверхностного знакомства с технологиями.

Основные вопросы

1. Расскажите о последнем Go-проекте: какую проблему решал сервис, какая была ваша зона ответственности?
2. Какой самый сложный технический выбор вы делали в этом проекте? Какие альтернативы рассматривали?
3. С какими нагрузками работал сервис: RPS, объем данных, latency, количество пользователей или сообщений?
4. Как была организована разработка: code review, тестирование, CI/CD, релизы, мониторинг?
5. Приведите пример production-инцидента, в расследовании которого вы участвовали. Как нашли причину?
6. Что в Go вам нравится, а что кажется неудобным или ограничивающим?

Сильные сигналы

- Кандидат говорит конкретно: метрики, ограничения, решения, последствия.
- Может объяснить не только что делал, но и почему.
- Осознает стоимость решений: complexity, maintainability, latency, reliability.

Риски

- Описывает опыт общими словами без деталей.
- Не может объяснить архитектуру сервиса, над которым работал.
- Приписывает себе командные решения, но не показывает личный вклад.

2. Go core и идиоматика

Темы для проверки

- packages, visibility, init;
- pointers, values, escape analysis на базовом уровне;
- slices, maps, arrays;
- interfaces и embedding;
- error handling;
- defer, panic, recover;
- generics;
- standard library;
- memory model на практическом уровне.

Вопросы по языку

1. Чем slice отличается от array? Что хранит slice header?
2. Что произойдет при append к slice, если capacity достаточно, и если capacity недостаточно?
3. Почему изменение одного slice может неожиданно изменить другой slice?
4. Можно ли читать из nil map? Можно ли писать в nil map?
5. Что происходит при обращении к отсутствующему ключу map?
6. Почему map в Go не потокобезопасна?
7. Когда стоит использовать pointer receiver, а когда value receiver?
8. Чем interface value с nil concrete value отличается от nil interface?
9. Как Go выбирает, реализует ли тип интерфейс?

10. Для чего нужен empty interface или any? Когда его использование ухудшает дизайн?
11. Как работает defer? В каком порядке выполняются несколько defer?
12. Что будет с аргументами defer: вычисляются сразу или в момент выполнения?
13. Когда допустимо использовать panic/recover в backend-сервисе?
14. Как вы обычно оборачиваете и сравниваете ошибки?
15. Чем errors.Is отличается от errors.As?
16. Когда уместны sentinel errors, а когда лучше typed errors?
17. Что такое zero value и почему это важная идея в Go?
18. Как generics изменили подход к utility-коду? Когда не стоит использовать generics?
19. Какие пакеты стандартной библиотеки вы используете чаще всего?
20. Что такое escape analysis и как она может влиять на производительность?

Практические мини-вопросы

Slice aliasing

```
a := []int{1, 2, 3, 4}
b := a[:2]
b = append(b, 99)
fmt.Println(a)
```

Вопросы:

- Что будет выведено?
- Почему?
- Как избежать неожиданного изменения исходного slice?

Ожидаемое направление ответа: кандидат понимает shared backing array и предлагает copy или full slice expression с ограничением capacity.

Nil interface

```
type MyError struct{}

func (e *MyError) Error() string { return "boom" }

func do() error {
    var err *MyError = nil
    return err
}

fmt.Println(do() == nil)
```

Вопросы:

- Что будет выведено?
- Почему error interface не равен nil?
- Как правильно вернуть отсутствие ошибки?

Ожидаемое направление ответа: interface содержит dynamic type и dynamic value; nil должен возвращаться как untyped nil.

3. Concurrency и context

Темы для проверки

- goroutines lifecycle;
- channels: buffered/unbuffered, close, select;

- sync.Mutex, RWMutex, WaitGroup, Once, Cond;
- atomic operations;
- context cancellation and deadlines;
- race conditions;
- worker pools;
- backpressure;
- graceful shutdown.

Основные вопросы

1. Чем goroutine отличается от OS thread?
2. Что может привести к goroutine leak?
3. Как обнаружить goroutine leak в production?
4. Когда использовать channel, а когда mutex?
5. Что произойдет при чтении из закрытого channel?
6. Что произойдет при записи в закрытый channel?
7. Кто должен закрывать channel: отправитель или получатель?
8. Для чего нужен select с default?
9. Как реализовать fan-out/fan-in?
10. Как ограничить количество одновременно выполняющихся задач?
11. Как корректно остановить worker pool?
12. Как работает context.Context и что нельзя хранить в context?
13. Чем context cancellation отличается от timeout/deadline?
14. Как передавать context в функции и методы?
15. Почему нельзя игнорировать ctx.Done() в долгих операциях?
16. Что такое data race? Как race detector помогает, а где его недостаточно?
17. Когда использовать sync/atomic вместо mutex?
18. Что такое backpressure и как его реализовать?
19. Как сделать graceful shutdown HTTP-сервера?
20. Как бы вы защитили сервис от лавинообразного роста goroutines?

Практический concurrency-сценарий

Нужно обработать 10 000 задач, каждая делает HTTP-запрос к внешнему сервису. Внешний сервис нестабилен, есть rate limit, запросы могут зависать.

Попросите кандидата описать решение:

- ограничение параллелизма;
- context timeout на запрос;
- retry policy с backoff;
- обработка частичных ошибок;
- сбор результатов;
- остановка при отмене;
- метрики и логирование;
- защита от goroutine leaks.

Сильный ответ включает worker pool или semaphore, context propagation, bounded queue, timeouts, idempotency considerations, observability и понятную стратегию ошибок.

4. Backend, API и работа с данными

HTTP/gRPC и API

1. Как вы проектируете REST API для нового ресурса?
2. Какие HTTP-коды вы используете для validation error, conflict, unauthorized, forbidden, not found?
3. Как организовать versioning API?
4. Как валидировать входные данные?
5. Где должна жить бизнес-логика: handler, service, repository?
6. Как вы обрабатываете request timeout?
7. Чем middleware отличается от handler wrapper?
8. Как реализовать idempotency для POST-запроса?
9. Когда выбирать gRPC вместо REST?
10. Какие проблемы возникают при backward-incompatible изменениях API?

Базы данных

1. Как устроена транзакция? Что такое commit и rollback?
2. Какие уровни изоляции транзакций вы знаете?
3. Что такое dirty read, non-repeatable read, phantom read?
4. Как избежать lost update?
5. Как работает optimistic locking?
6. Как выбрать индекс для SQL-запроса?
7. Почему индекс может не использоваться?
8. Как анализировать медленный SQL-запрос?
9. Чем INNER JOIN отличается от LEFT JOIN?
10. Что такое N+1 problem и как ее исправить?
11. Как проектировать миграции без downtime?
12. Как хранить и обрабатывать nullable-поля в Go?
13. Когда использовать SQL, а когда NoSQL?
14. Как организовать pagination: offset vs cursor?
15. Как вы проектируете repository layer?

Reliability

1. Что такое timeout, retry и circuit breaker?
2. Почему retries могут ухудшить инцидент?
3. Что такое idempotency key?
4. Как обрабатывать partial failure?
5. Как проектировать взаимодействие с внешним API?
6. Что логировать при ошибке, а что нельзя логировать?
7. Какие метрики нужны для HTTP-сервиса?
8. Как понять, что сервис деградирует до полной поломки?

5. Практическая задача

Выберите одну задачу в зависимости от фокуса вакансии. Для Middle+ важно не только получить работающий код, но и увидеть ход мыслей, декомпозицию, обработку ошибок и тестируемость.

Вариант А: In-memory rate limiter

Задача: реализовать in-memory rate limiter для API по userID.

Требования:

- лимит N запросов за окно времени T;
- потокобезопасность;
- метод `Allow(userID string) bool`;
- очистка старых записей;
- возможность написать unit tests;
- обсуждение поведения при большом количестве пользователей.

Вопросы после реализации:

1. Какой алгоритм вы выбрали: fixed window, sliding window, token bucket? Почему?
2. Какие edge cases есть на границе окна?
3. Как избежать бесконечного роста map?
4. Как сделать реализацию распределенной?
5. Какие метрики вы бы добавили?

Критерии оценки:

- корректная синхронизация;
- простая и понятная структура данных;
- тесты на границы окна;
- отсутствие глобального состояния;
- понимание ограничений in-memory решения.

Вариант В: Concurrent fetch with timeout

Задача: написать функцию, которая параллельно запрашивает данные по списку URL и возвращает успешные ответы.

Пример сигнатуры:

```
func FetchAll(ctx context.Context, client *http.Client, urls []string, limit int) ([]Result, error)
```

Требования:

- ограничить параллелизм параметром `limit`;
- уважать отмену context;
- не допустить goroutine leaks;
- вернуть частичные результаты;
- корректно обработать HTTP errors и timeouts;
- написать table-driven tests с mock server.

Вопросы после реализации:

1. Как остановятся goroutines при отмене context?
2. Что произойдет, если один URL зависнет?
3. Как сохранить порядок результатов?

4. Как добавить retry?
5. Как логировать ошибки без утечки персональных данных?

Вариант C: Transactional service method

Задача: спроектировать сервисный метод перевода денег между счетами.

Требования:

- проверить существование счетов;
- проверить достаточность средств;
- выполнить debit и credit атомарно;
- защититься от concurrent transfers;
- сохранить audit log;
- вернуть доменные ошибки;
- обсудить тестирование.

Вопросы после реализации:

1. Где должна начинаться транзакция?
2. Какой уровень изоляции выбрать?
3. Как избежать deadlock?
4. Как сделать операцию идемпотентной?
5. Как обработать сбой после commit, но до ответа клиенту?

6. System design light

Сценарий

Спроектируйте сервис уведомлений, который принимает события от других сервисов и отправляет email, SMS и push-уведомления.

Что попросить кандидата раскрыть

1. API приема событий.
2. Валидация и нормализация данных.
3. Очередь или брокер сообщений.
4. Хранение статуса уведомлений.
5. Retry и dead-letter queue.
6. Idempotency и дедупликация.
7. Rate limits внешних провайдеров.
8. Шаблоны уведомлений.
9. Observability.
10. Graceful degradation.
11. Horizontal scaling.
12. Безопасность и персональные данные.

Уточняющие вопросы

1. Что произойдет, если провайдер SMS недоступен 30 минут?
2. Как не отправить одно уведомление дважды?

3. Как гарантировать, что событие не потеряется?
4. Какой `delivery guarantee` выбрать: `at-most-once`, `at-least-once` или `exactly-once`?
5. Какие таблицы или коллекции нужны?
6. Какие индексы понадобятся?
7. Как мониторить задержку доставки?
8. Как отделить критичные уведомления от маркетинговых?
9. Как безопасно хранить шаблоны и пользовательские данные?
10. Как тестировать такую систему?

Сильный ответ

Кандидат не пытается сразу построить чрезмерно сложную систему, а уточняет требования, выделяет критичные потоки, объясняет `trade-off` между простотой, надежностью и стоимостью. В ответе должны появиться `idempotency`, `retries`, `DLQ`, `outbox` или аналогичный паттерн для надежной публикации событий, метрики, `structured logs` и понятная модель данных.

7. Тестирование и качество

Вопросы

1. Как вы обычно пишете `unit tests` в Go?
2. Что такое `table-driven tests` и почему они популярны в Go?
3. Как тестировать код, который зависит от времени?
4. Как тестировать HTTP handler?
5. Как тестировать интеграцию с базой данных?
6. Когда использовать `mocks`, `fakes` и `testcontainers`?
7. Как проверять `race conditions`?
8. Что должно быть в CI pipeline для Go-сервиса?
9. Какой уровень покрытия тестами вы считаете полезным?
10. Как вы относитесь к `snapshot tests`?

Практическое обсуждение

Дайте кандидату небольшой фрагмент плохо тестируемого кода: прямой вызов `time.Now`, глобальный HTTP client, работа с БД внутри handler. Попросите объяснить, как сделать код тестируемым.

Ожидаемые идеи:

- `dependency injection`;
- интерфейсы на границах;
- `clock abstraction`;
- `httptest`;
- `transaction rollback` в интеграционных тестах;
- разделение `handler/service/repository`.

8. Performance и profiling

Вопросы

1. Как понять, что Go-сервис упирается в CPU, память, БД или сеть?
2. Как использовать pprof?
3. Какие профили вы знаете: cpu, heap, goroutine, mutex, block?
4. Что такое allocation и почему лишние allocations важны?
5. Когда стоит оптимизировать код?
6. Как benchmark отличается от unit test?
7. Как написать корректный benchmark в Go?
8. Что может привести к высокому GC pressure?
9. Как искать memory leak в Go?
10. Какие типичные причины latency spikes?

Сильные сигналы

- Кандидат начинает с измерений, а не с догадок.
- Умеет пользоваться pprof, metrics и tracing.
- Понимает, что преждевременная оптимизация может ухудшить дизайн.

9. Observability и эксплуатация

Вопросы

1. Какие structured logs нужны в HTTP-сервисе?
2. Что должно быть в request ID или trace ID?
3. Какие RED/USE-метрики вы знаете?
4. Какие alerts вы бы настроили для backend-сервиса?
5. Чем symptoms-based alerts лучше cause-based alerts?
6. Как логировать ошибки внешних API?
7. Как связаны tracing и logs?
8. Что должно быть в health check?
9. Чем readiness отличается от liveness?
10. Как организовать graceful shutdown в Kubernetes?

10. Security

Вопросы

1. Как хранить пароли пользователей?
2. Почему нельзя логировать access tokens и персональные данные?
3. Чем authentication отличается от authorization?
4. Как устроен JWT? Какие у него риски?
5. Что такое CSRF и когда он актуален?
6. Что такое SQL injection и как от него защищаться?
7. Как валидировать входные данные?
8. Как хранить secrets для сервиса?
9. Как ограничить blast radius при компрометации сервиса?

10. Какие security checks можно добавить в CI?

11. Code review

Формат

Покажите кандидату pull request или фрагмент кода с намеренными проблемами:

- business logic в HTTP handler;
- отсутствие context propagation;
- бесконечные retries;
- игнорирование ошибок;
- race condition;
- нет timeout у HTTP client;
- SQL-запрос без индекса;
- логирование чувствительных данных;
- непроверенные edge cases.

Вопросы

1. Какие проблемы вы видите?
2. Какие замечания обязательны к исправлению, а какие можно оставить как улучшение?
3. Как бы вы сформулировали комментарий в code review?
4. Какие тесты нужно добавить?
5. Что можно упростить без изменения поведения?

Сильный ответ

Кандидат приоритизирует риски, не заикливаясь на стиле, объясняет влияние проблемы на пользователей или эксплуатацию, предлагает конкретные исправления и умеет коммуницировать без агрессии.

12. Поведенческий блок

Вопросы

1. Расскажите о случае, когда вы не согласились с техническим решением команды. Что сделали?
2. Когда вы в последний раз сильно упростили решение? Что помогло увидеть простоту?
3. Приведите пример, когда вы ошиблись в production. Как исправляли последствия?
4. Как вы принимаете решение между быстрым fix и правильным refactoring?
5. Как вы помогли менее опытному разработчику?
6. Что для вас хороший code review?
7. Как вы действуете, если задача плохо сформулирована?
8. Как вы сообщаете менеджеру или команде о техническом риске?
9. Что вы делаете, если не успеваете закончить задачу в ожидаемый срок?
10. Как вы поддерживаете качество в условиях давления на сроки?

13. Матрица оценки

Оценивайте каждый блок по шкале 1-4.

Балл	Интерпретация
1	Знания фрагментарные, много критичных пробелов, нужен постоянный контроль
2	Уровень ближе к Middle-, решает типовые задачи, но слабо видит риски
3	Уверенный Middle/Middle+, самостоятельно решает production-задачи
4	Сильный Middle+/Senior-, системно мыслит, предвидит проблемы, усиливает команду

Компетенции

Компетенция	Вес	Что считать хорошим результатом
Go core и идиоматика	20%	Понимает язык, ошибки, interfaces, slices/maps, стандартную библиотеку
Concurrency	20%	Уверенно работает с goroutines, channels, context, races
Backend и данные	20%	Проектирует API, транзакции, надежную работу с внешними системами
Практическое кодирование	20%	Пишет простой, корректный, тестируемый код
Архитектура и эксплуатация	15%	Видит trade-off, observability, reliability, scaling
Коммуникация и ownership	5%	Ясно объясняет решения, принимает ответственность

Decision guide

- Strong hire: в большинстве блоков 3-4, нет критичных пробелов в concurrency, данных и production-надежности.
- Hire: стабильные 3, отдельные слабые зоны компенсируются опытом и обучаемостью.
- Lean no hire: много ответов на уровне теории, слабая практика с context, транзакциями или тестами.
- No hire: не может объяснить базовые конструкции Go, не видит race conditions, игнорирует ошибки и эксплуатационные риски.

14. Красные флаги

- Не понимает nil interface и error handling в Go.
- Не умеет объяснить lifecycle goroutine и риски goroutine leaks.
- Предлагает unbounded goroutines для внешних запросов.
- Игнорирует context cancellation.
- Использует panic для обычных бизнес-ошибок.
- Не пишет тесты или считает их необязательными.
- Не умеет объяснить транзакции и конкурентное обновление данных.
- Считает retries универсальным решением без timeout и backoff.
- Не различает authentication и authorization.
- Не может рассказать о собственных ошибках и выводах.

15. Вопросы кандидата к интервьюеру

Оставьте 5-10 минут на вопросы кандидата. Хорошие вопросы кандидата тоже являются сигналом зрелости.

На что обратить внимание:

- спрашивает о домене, архитектуре, нагрузке, команде;
- интересуется качеством процессов, on-call, observability;
- уточняет ожидания от роли и зоны ответственности;
- задает вопросы о техническом долге и планах развития.

16. Итоговый шаблон заметок интервьюера

Кандидат:

Дата:

Интервьюеры:

Краткое резюме:

Go core:

Оценка 1-4:

Заметки:

Concurrency:

Оценка 1-4:

Заметки:

Backend/API/DB:

Оценка 1-4:

Заметки:

Практическая задача:

Оценка 1-4:

Заметки:

System design:

Оценка 1-4:

Заметки:

Коммуникация и ownership:

Оценка 1-4:

Заметки:

Сильные стороны:

Риски:

Рекомендация:

Strong hire / Hire / Lean no hire / No hire

Аргументация: